# Continual Learning & Memory Models

## COMS 6998

**Week #4: February 18, 2026**

**Paper title: Recurrent Memory Transformer**

# Background: Transformer's Self attention

- **Main Benefit:**
  - Rich contextual representation
  - Combines global + local info into single representation
- **Key problem:**
  - Global feature "blurring" among tokens
  - Hard to access a "specific" feature due to same representation
  - Poor scaling w.r.t token lengths → harder to maintain long range dependencies



**Author:** Samuel Lee (sl5806)

# Previous Memory Representation (Related Works)

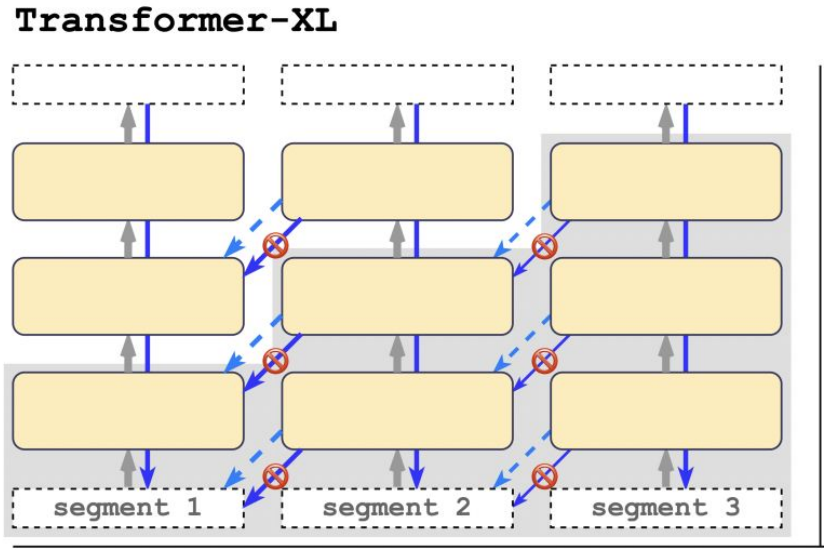
- Earlier Approaches
  - LSTM
  - BPTT
- Recurrent External Memory Approaches
  - NTM
  - MANN
  - DNC
- Transformer Family
  - Transformer-XL → hidden-state representation cache recurrence
  - Memory Transformer → special memory tokens
  - Stair-case Transformer → combines segment level + depth level recurrence
  - Longformer / GMAT / ETC / BigBird



**Author:** Samuel Lee (sl5806)

# Main Comparison: Transformer-XL

- **Current Solution:**
  Transformer-XL
  - Integrates segment-level recurrence
  - Caches the hidden state representations
- But…
  - Memory scaling w.r.t segments by *m x N* vectors
  - Mixing sequence tokens with memory segments




**GOAL:** Can we design a transformer that stores global info explicitly AND better handle long sequences WHILE being more memory efficient?



**Author:** Samuel Lee (sl5806)

# Proposal

**GOAL:** Can we design a transformer that stores global info explicitly AND better handle long sequences WHILE being more memory efficient?

- Why don't we add specialized memory tokens before + after the sequence tokens?

  ***[mem] + sequence_tokens + [mem]***

- Make this recur between segments

  ***[mem] + seq1 + [mem]*** ***[mem] + seq2 + [mem] …***

- No need to modify the Transformer backbone itself

Learn how to use "memory" via memory tokens of (compressed) segments



**Author:** Samuel Lee

# Recurrent Memory Transformer (RMT) - Design Goal & Setup

***Goal:*** Enable long-context processing without modifying Transformer layers.

**Setup:**

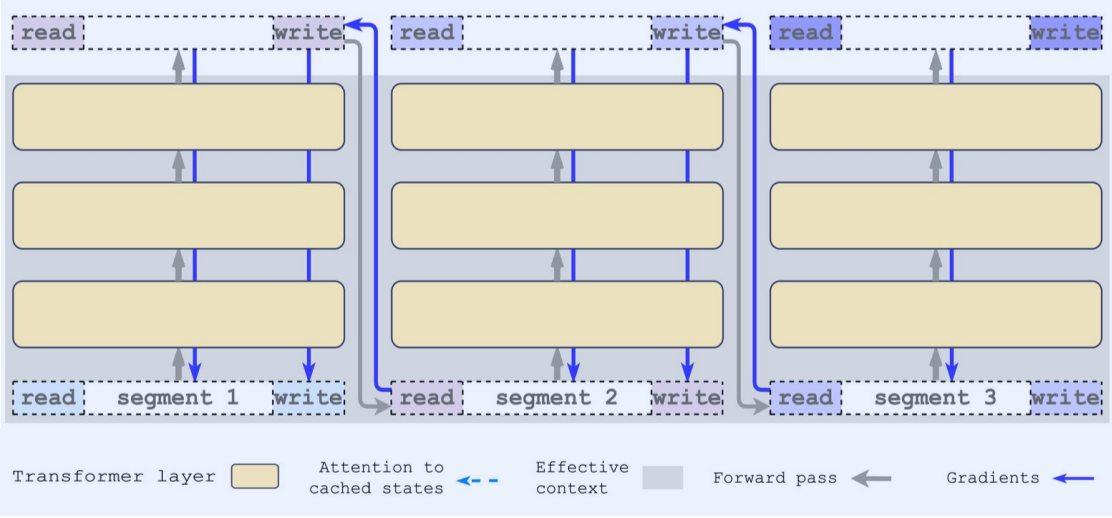
- Splitting long sequences into **segments** $\tau$
- Processing each segment sequentially
- Introducing learnable **memory tokens**

**Notations:**

- $H_{\tau}^{0}$: token representations of segment $\tau$
- $m$: number of memory tokens
- $N$: number of Transformer layers

**Key Idea:** Creating recurrence using only token-level operations.

- RMT treats memory as just extra tokens
- Total extra tokens per segment = $2m$



# RMT - Recurrent Memory Mechanism

## (1) Augmented Input

- $H_\tau^0$ : segment token embeddings
- $H_\tau^{mem}$ : memory tokens

$$\tilde{H}_\tau^0 = [\underbrace{H_\tau^{mem}}_{\text{read}} \circ H_\tau^0 \circ \underbrace{H_\tau^{mem}}_{\text{write}}]$$

**Why Two Memory Blocks?**

- **Read Memory:** Segment tokens can attend to it, so it provides access to information stored from previous segments.
- **Write Memory:** Positioned after all segment tokens, it can attend to the entire segment and summarize global information.

Design ensures that memory tokens can both (i) read past context and (ii) write updated information without violating the causal attention mask.

## (2) Transformer Preprocessing

Passing this augmented sequence through a standard N-layer Transformer:

$$\bar{H}_\tau^N = \text{Transformer}(\tilde{H}_\tau^0)$$

Then splitting the output into updated read memory, segment tokens, and write memory.

$$[H_\tau^{read} \circ H_\tau^N \circ H_\tau^{write}]$$

## (3) Recurrence Rule

Write memory at the end of segment $\tau$ becomes the memory available at the beginning of segment $\tau$+1.

$$H_{\tau+1}^{mem} := H_\tau^{write}$$

$$\tilde{H}_{\tau+1}^0 = [H_{\tau+1}^{mem} \circ H_{\tau+1}^0 \circ H_{\tau+1}^{mem}]$$

This creates recurrence purely through token passing, without modifying the Transformer layers.

# RMT - Training & Gradient Flow

## *Memory Chain Across Segments*

- Forward recurrence:

$$H_{\tau+1}^{mem} := H_{\tau}^{write}$$

- This forms a persistent memory chain:

$$H_{\tau}^{write} \rightarrow H_{\tau+1}^{mem} \rightarrow H_{\tau+1}^{write}$$

- Memory behaves like a learned compressed recurrent state

## *Gradient Flow Through Memory*

- Memory connection remains part of computational graph
- Gradients from future segments supervise how earlier segments wrote memory
- Model learns how to write useful information into memory

## *Backpropagation Through Time*

- Unroll length: 0–4 segments (hyperparameter)
- Larger unroll → stronger supervision of memory
- Higher GPU memory cost & potential instability

## *Why this design works?*

- Fixed memory size $m \rightarrow$ compact representation
- Memory refined through all $N$ Transformer layers
- Compatible with pretrained Transformers
- Recurrence without architectural modification
- Dedicated memory tokens prevent mixing of global & local information


Berkay Baris Turan (bt2686) - Author

# Experiments: what RMT is tested on

**Baselines / comparisons**

- Transformer baseline (no recurrence)
- Transformer-XL (segment recurrence via cached hidden states; typically no BPTT across segments)
- RMT variants: memory size m + BPTT unroll (0–3 in main results)

**Algorithmic tasks** (stress-test memory)

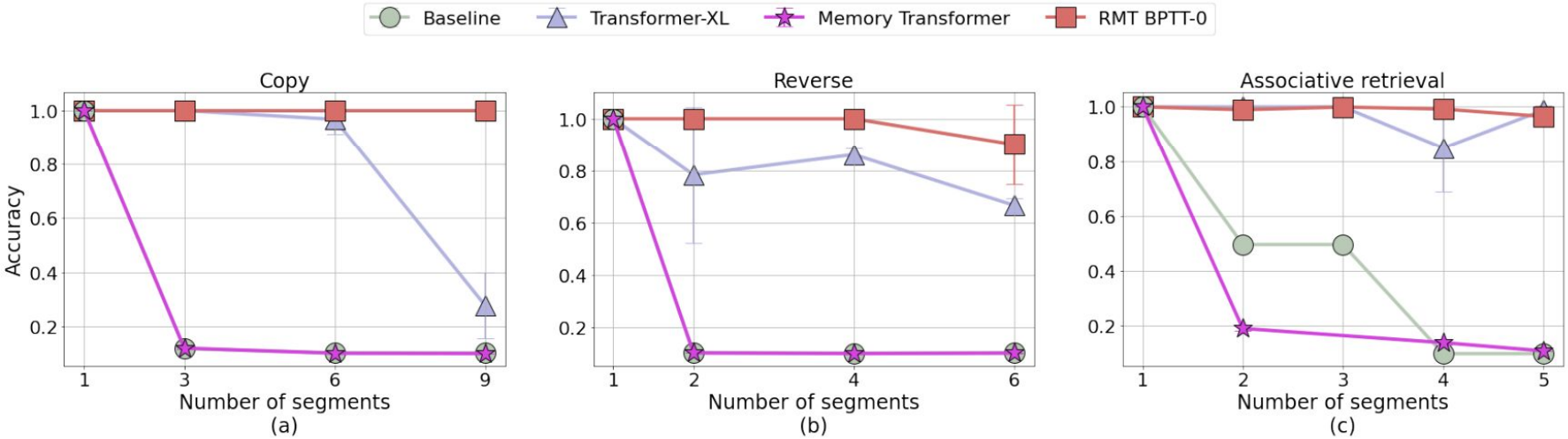
- Copy / Reverse / Associative Retrieval across multiple segments
- Quadratic equations: answer only appears in the last segment

**NLP tasks**

- Language modeling: WikiText-103 (word-level) + enwik8 (character-level)
- Long-document classification: Hyperpartisan news (add RMT memory to pretrained encoders)

Setup: long sequences split into segments; recurrence via memory tokens; BPTT unroll = how many segments get gradient

# Results: long-range performance with “smaller” memory




Memory/cache size is equal to the length of a segment for both models

# Results: long-range performance with “smaller” memory

| MODEL | MEMORY | SEGMENTS | ACC$_{\pm \text{STD}}$ |
|---|---|---|---|
| BASELINE | 0 | 1 | $0.99 \pm 0.01$ |
| TRANSFORMER-XL | 30 | 6 | $0.93 \pm 0.02$ |
| RMT | 30 | 6 | $0.99 \pm 0.002$ |

RMT outperforms Tr-XL! (to no surprise)

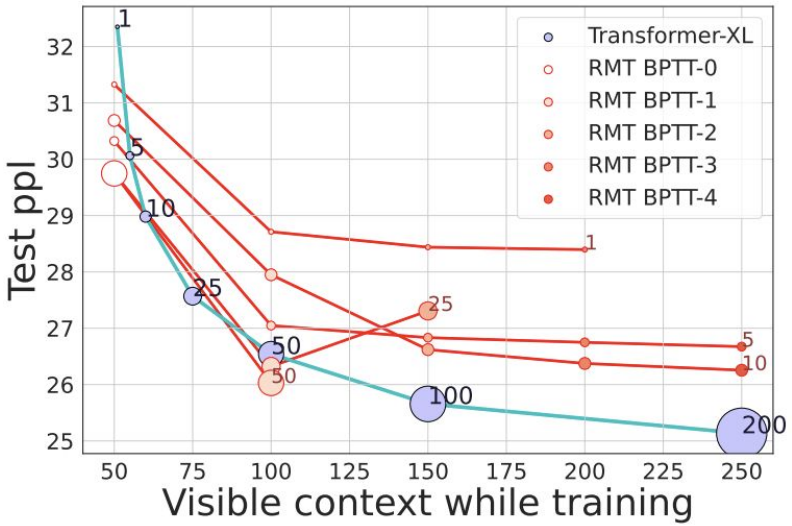
# Results: long-range performance with “smaller” memory

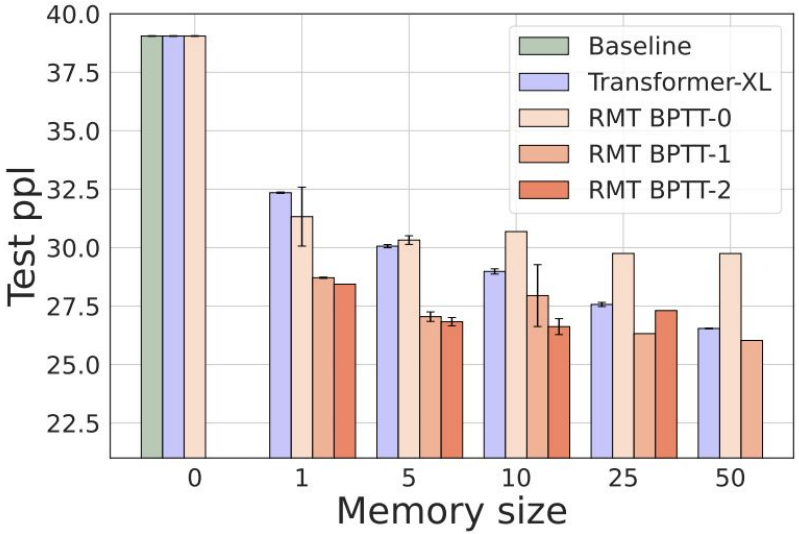
| MODEL | MEMORY | SEGMENT LEN | PPL $\pm$ STD |
|---|---|---|---|
| TR-XL (PAPER) | 150 | 150 | 24.0 |
| BASELINE | 0 | 150 | 29.95 $\pm$ 0.15 |
| MEMTR | 10 | 150 | 29.63 $\pm$ 0.06 |
| TR-XL (OURS) | 150 | 150 | 24.12 $\pm$ 0.05 |
| TR-XL | 25 | 150 | 25.57 $\pm$ 0.02 |
| TR-XL | 75 | 150 | 24.68 $\pm$ 0.01 |
| RMT BPTT-3 | 10 | 150 | 25.04 $\pm$ 0.07 |
| RMT BPTT-2 | 25 | 150 | 24.85 $\pm$ 0.31 |
| TR-XL + RMT | 75+5 | 150 | 24.47 $\pm$ 0.05 |
| TR-XL + RMT | 150+10 | 150 | **23.99** $\pm$ 0.09 |
| BASELINE | 0 | 50 | 39.05 $\pm$ 0.01 |
| TR-XL | 100 | 50 | **25.66** $\pm$ 0.01 |
| TR-XL | 50 | 50 | 26.54 $\pm$ 0.01 |
| TR-XL | 25 | 50 | 27.57 $\pm$ 0.09 |
| TR-XL | 10 | 50 | 28.98 $\pm$ 0.11 |
| RMT BPTT-1 | 1 | 50 | 28.71 $\pm$ 0.03 |
| RMT BPTT-3 | 10 | 50 | 26.37 $\pm$ 0.01 |

Language modeling on WikiText-103

Segment Length: # of tokens in each segment

# Results: long-range performance with “smaller” memory




Visible Context: Number of tokens a token attends to in self-attention (including [mem])
Note: Marker size corresponds to memory size

# Implications + Future Work

## What the experiments suggest

Explicit recurrent memory can preserve long-range info better than relying only on cached hidden states

Small, learned state can be competitive: memory is *compressed*, not a growing cache

Hybrid design is promising: cache for local detail + recurrent memory for global state

## Limitations

BPTT unroll improves performance but increases training memory/compute (needs checkpointing/truncation)

Fixed segmenting and fixed memory size may be suboptimal across domains

## Future Work

Adaptive memory: dynamic number of memory tokens; hierarchical (short/medium/long) memory

Richer benchmarks: more long-document QA/summarization, multi-doc reasoning, and retrieval-augmented hybrids

Interpretability: track what information is stored in memory over time; failure analysis when memory saturates

# Summary

The paper proposes "Recurrent Memory Transformer" (RMT), a modification to the Transformer architecture that introduces special memory tokens to the input sequence. These tokens are processed alongside the standard sequence tokens and then passed to the next segment to enable segment-level recurrence. Unlike Transformer-XL, which caches hidden states with a stop-gradient operation, RMT employs Backpropagation Through Time (BPTT) to train these memory tokens across segments. The authors evaluate the model on synthetic algorithmic tasks (Copy, Reverse, Associative Retrieval, Quadratic Equations) and language modeling benchmarks (WikiText-103, enwik8). They claim the model outperforms Transformer-XL on tasks requiring long-term dependency retention (algorithmic tasks) and achieves comparable performance on language modeling with potentially smaller memory footprints.

# Strengths

**Simplicity of Implementation:** The proposed method is conceptually simple and requires no fundamental changes to the Transformer attention mechanism itself, as it operates by manipulating the input/output sequences. It has the potential to be a plug and play method for many models.

**Clearity:** The paper is easy to understand and well-written. Experiments and hyperparameters in details to ensure reproducibility. Open-sourced.

**Interpretability:** The visualization of attention maps (Figure 6) provides some insight into how the model utilizes memory tokens for read/write operations, distinguishing it from the caching mechanism of Transformer-XL.

# Weakness

1. **Results: Missing SOTA baselines.** The results of language modeling (table 2) contains only Transformer-XL, Memory Transformer and RMT. More SOTA methods, such as Compressive Transformer (2019), Longformer (2020) and Memformer (2020), are cited in related works but not evaluated. However, the paper listed results of SOTA models for a downstream task, hyperpartisan news detection in table 3. Note that table 3 shows a classification task and it is different from generative task such as language modeling. It might be challenged for “cherry picking” and lack of rigorous comparison in the primary evaluation.

Table 2: **Language modeling on WikiText-103.** Average perplexity for the best performed variations of RMT models reported (see full results in Appendix A.5). Underlined values show Tr-XL and RMT models with close results. RMT models with smaller memory sizes achieve similar scores to Tr-XL models with larger memory. Combination of cache with recurrent memory (Tr-XL + RMT) shows the best performance.

| Model | Memory | Segment Len | PPL±std |
|---|---|---|---|
| Tr-XL (paper) | 150 | 150 | 24.0 |
| Baseline | 0 | 150 | 29.95 ± 0.15 |
| MemTr | 10 | 150 | 29.63 ± 0.06 |
| Tr-XL (ours) | 150 | 150 | 24.12 ± 0.05 |
| Tr-XL | 25 | 150 | 25.57 ± 0.02 |
| Tr-XL | 75 | 150 | 24.68 ± 0.01 |
| RMT BPTT-3 | 10 | 150 | 25.04 ± 0.07 |
| RMT BPTT-2 | 25 | 150 | 24.85 ± 0.31 |
| Tr-XL + RMT | 75+5 | 150 | 24.47 ± 0.05 |
| Tr-XL + RMT | 150+10 | 150 | **23.99** ± 0.09 |
| Baseline | 0 | 50 | 39.05 ± 0.01 |
| Tr-XL | 100 | 50 | **25.66** ± 0.01 |
| Tr-XL | 50 | 50 | 26.54 ± 0.01 |
| Tr-XL | 25 | 50 | 27.57 ± 0.09 |
| Tr-XL | 10 | 50 | 28.98 ± 0.11 |
| RMT BPTT-1 | 1 | 50 | 28.71 ± 0.03 |
| RMT BPTT-3 | 10 | 50 | 26.37 ± 0.01 |

Table 3: **Hyperpartisan news detection.** Models starting with RMT are taken from HuggingFace Transformers and augmented with 10 memory tokens and recurrence before fine-tuning. Train/valid/test split as in (Beltagy et al., 2020) and metric is F1.

| Model [input size] | Number of Segments 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| Big Bird [4096] (Zaheer et al., 2020) | 92.20 | | | |
| Longformer [4096] (Beltagy et al., 2020) | 94.80 | | | |
| Graph-roberta [512x100] (Xu et al., 2021) | 96.15 | | | |
| ERNIE-DOC-Large [640] (Ding et al., 2021) | 96.60 | | | |
| ERNIE-Sparse [4096] (Liu et al., 2022) | 92.81 | | | |
| RMT bert-base-case [512] | 91.60 | 94.12 | 93.06 | 94.34 |
| RMT roberta-base [512] | 94.87 | **97.20** | **96.72** | **98.11** |
| RMT deberta-v3-base [512] | 94.17 | 96.78 | 94.80 | 94.80 |
| RMT t5-base [512] | **94.99** | 95.32 | 96.12 | 97.20 |

# Weakness

1. **Results: Missing SOTA baselines.**
2. **Training Efficiency and Scalability.**
   a. The reliance on Backpropagation Through Time (BPTT) over multiple segments significantly increases memory consumption and computational cost compared to Transformer-XL's stop-gradient cache mechanism. The authors explicitly admit that increasing the BPTT unroll depth is "computationally expensive and requires a lot of GPU RAM".
   b. The paper acknowledges "instabilities and out-of-memory issues" during training with deeper BPTT unrolls. This suggests that the model struggles to scale to longer contexts or larger model sizes, which contradicts the paper's goal of handling long-term dependencies. If the model is unstable and memory-hungry on relatively small benchmarks like WikiText-103, its scalability to modern Large Language Models (LLMs) with billions of parameters is highly questionable.
   c. While the authors claim RMT requires "less memory", this refers only to inference state size. It conveniently ignores the massive memory overhead required for storing gradients across segments during training.

# Weakness

1. **Results: Missing SOTA baselines.**
2. **Training Efficiency and Scalability.**
3. **Experiment Setting.** The paper shows a lot of its evaluation to synthetic algorithmic tasks such as Copy, Reverse, and Quadratic Equations. While the paper effectively demonstrates the model's capacity for memorization, the capability of reasoning or the utilization of memory of the model remains questionable. Also, the metric is simply PPL which might be insufficient.
    a. In the context of 2022, this experimental design was acceptable and aligned with the prevailing standards, as the field was still exploring the fundamental mechanics of long-term dependencies。
    b. But in the context of 2026, it would be problematic. The community has shifted towards more rigorous benchmarks like "Needle In A Haystack", which are specifically designed to test precise information retrieval and usage over massive contexts. The reliance on PPL and synthetic tasks fails to reveal potential issues such as "lossy compression" or "hallucination" that are critical in modern Large Language Model (LLM) applications.

# Conclusion

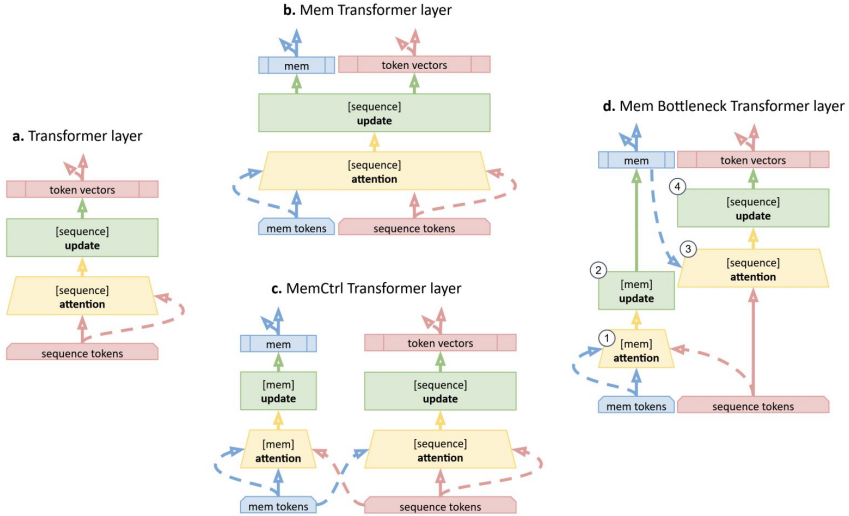
- Quality: 3 - Good
- Clarity: 4 - Excellement
- Significance: 3 - Good
- Originality: 2 - Fair

- **Overall: 4 - Borderline accept**: Technically solid paper where reasons to accept outweigh reasons to reject, e.g., limited evaluation.

- Confidence: 4 - You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

# Before the RMT

- **MemTransformer: Memory tokens (Burtsev et al., 2020) to**
  - (b) Add memory tokens for non-local representations
  - (c) Dedicated layer for memory updates
  - (d) Bottlenecks for global info




MemCtrl memory update:

$$A^{mem} = LN(X^{mem} + MH^{mem}(X^{mem}, X^{mem+seq}, X^{mem+seq})),$$
$$H^{mem} = LN(A^{mem} + FF^{mem}(A^{mem})).$$

MemCtrl sequence update:

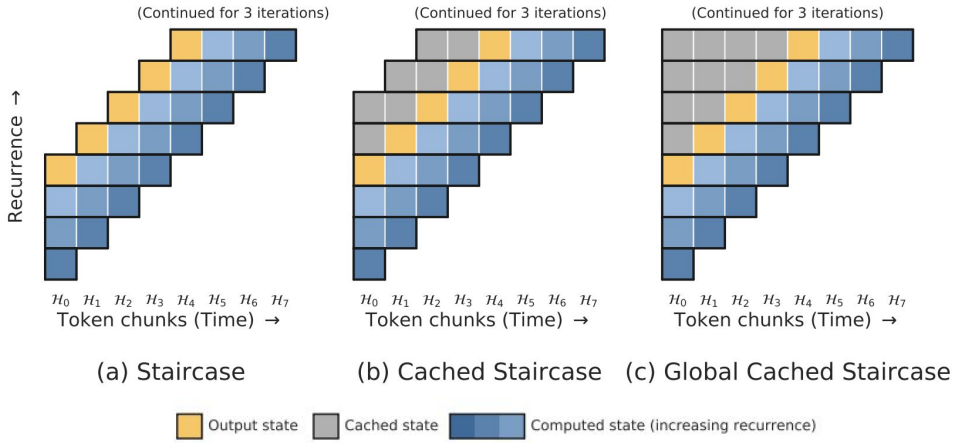
$$A^{seq} = LN(X^{seq} + MH^{seq}(X^{seq}, X^{mem+seq}, X^{mem+seq})),$$
$$H^{seq} = LN(A^{seq} + FF^{seq}(A^{seq})).$$



**Archeologist:** Brian Siegel

# Before the RMT

- **Staircase Transformer (Ju et al., 2021)**
- Introduces Staircase attention to create stepwise recurrence in time & depth
  - Like traditional self-attention, process tokens in parallel
  - But unlike self-attention, operate across sequence in time recurrently processing input by adding backwards and forward token in each step




**Method:**

- Ingest C input tokens in smaller chunks along set of multi-vector memory tokens
- Each step moves ahead in time using the same Transformer core to get recurrence through weight sharing and recurrence with sliding window
- Caching decouples context and recurrence

**Archeologist:** Brian Siegel

# After the RMT

- **ARMT ( Rodkin et al.,2025)**
- Replaces RMT's flat memory with a structured associative matrix
- Every memory tokens write to this matrix and token can query and retrieve specific facts across 50M+ token contexts.

- **Limitation**: Segment-level recurrence creates hard boundaries that hurt language modeling , ARMT inherits this fundamental weakness from RMT and does not fix it.

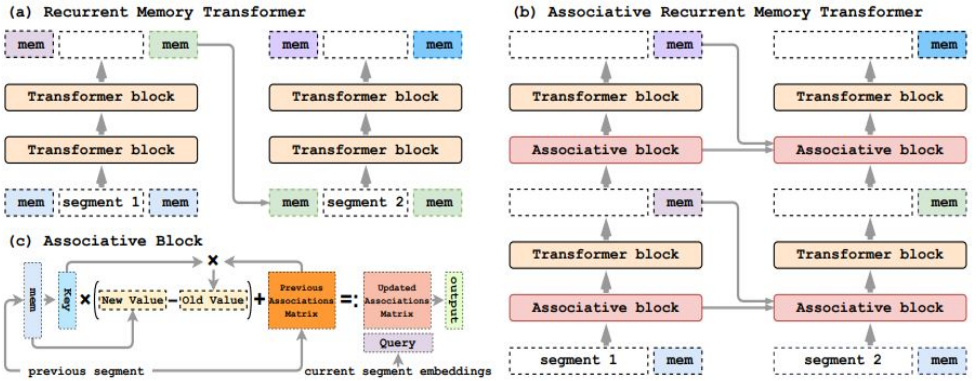



Figure 1: **ARMT augments the transformer's layers with associative memory.** (a) RMT architecture. (b) ARMT adds associative memory processing to each layer. (c) Associative memory is updated with layerwise memory representations.

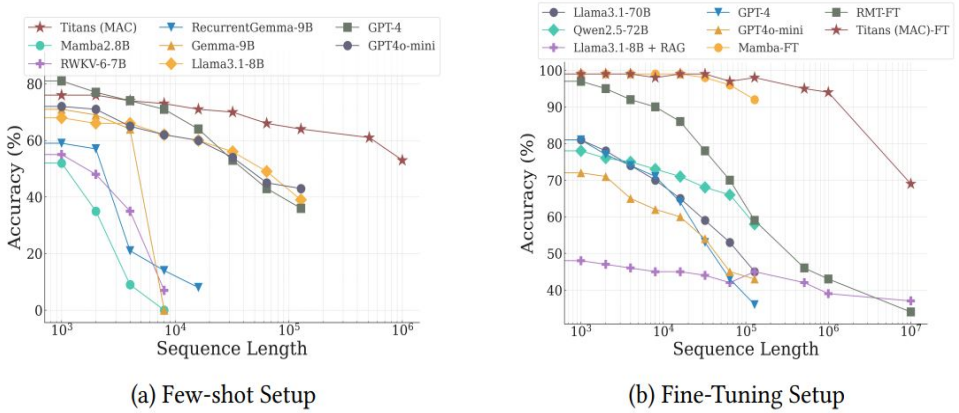
**Woking:** Every token queries the associative matrix before attention, self-attention then runs on these enriched tokens, and finally only the memory tokens update the associative matrix using erase-then-write.



**Archeologist:** Vedik Agarwal

# After the RMT

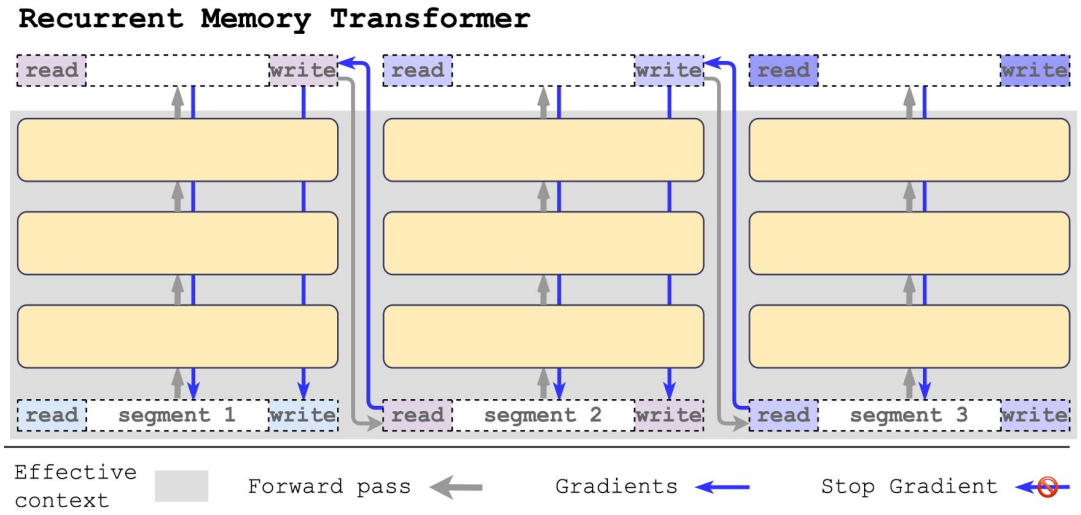
- **Titans: Learning to Memorize at Test Time (Behrouz et al. , 2025)**
- Introduces the MAC (Memory as context) architecture, which replaces the standard attention with a Neural Memory Module
- Unlike ARMT, which uses a mathematical "Matrix Update," Titans uses gradient-based updates at test time. It literally "learns" as it reads




(a) Few-shot Setup (b) Fine-Tuning Setup

- Uses a sophisticated "surprise-based" gating mechanism to decide which old information to overwrite and which new information is worth learning
- Outperforms GPT-4 on BABILong, scales to 2M+ token contexts

# Adaptive Memory Slot Allocation




What RMT provides:

- Enable BPTT across segments via memory tokens
- Fixed-sized memory tokens
- Sparsity level of memory token activations cannot be adjusted

What if the number of segments grows large?

- Information from multiple segments is compressed into a fixed-size memory token
- Information bottleneck
- Overwriting of previously stored content

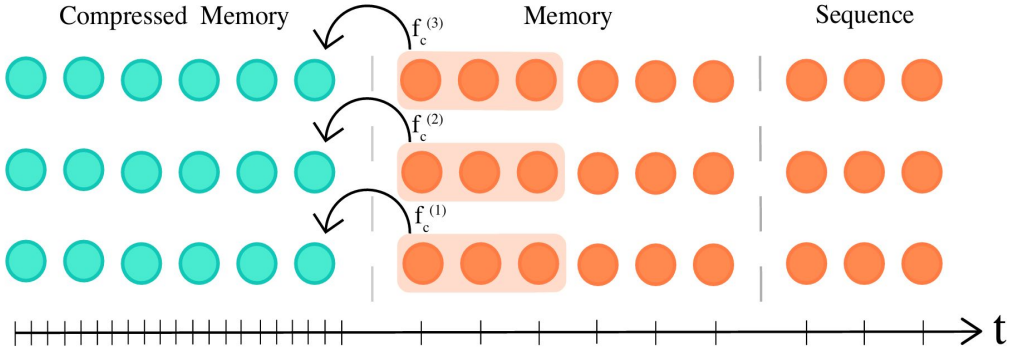


**Researcher 1:** Binyan Sun

# Compressive Transformers (2019)

Solved the memory problem of TransformerXL:

- ➢ Extends Transformer-XL memory
- ➢ Compressed old hidden states instead of discarding them
- ➢ Introduces multi-scale memory (fine + coarse)
- ➢ Successfully improves long-range modeling




**Algorithm 1** Compressive Transformer

At time zero

1: $\mathbf{m_0} \leftarrow \mathbf{0}$ // Initialize memory to zeros $(l \times n_m \times d)$
2: $\mathbf{cm_0} \leftarrow \mathbf{0}$ // Initialize compressed memory to zeros $(l \times n_{cm} \times d)$

At time t

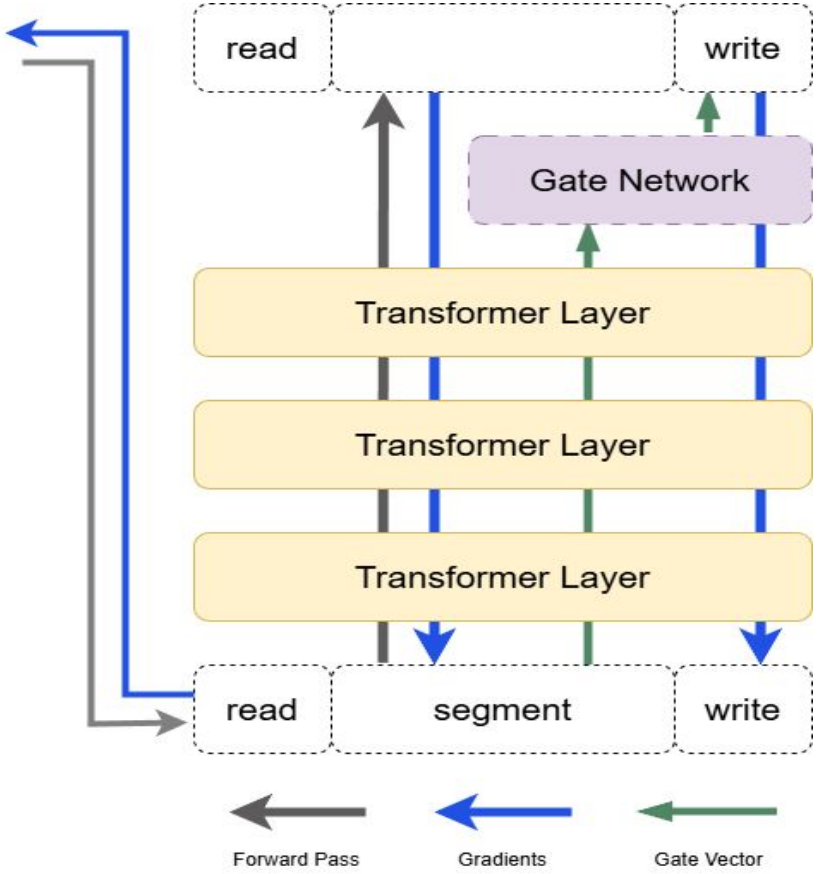
3: $\mathbf{h}^{(1)} \leftarrow \mathbf{x}\mathbf{W}_{\mathbf{emb}}$ // Embed input sequence$(n_s \times d)$
4: **for** layer $i = 1, 2, \ldots, l$ **do**
5: $\mathbf{mem}^{(\mathbf{i})} \leftarrow \text{concat}(\mathbf{cm}_\mathbf{t}^{(\mathbf{i})}, \mathbf{m}_\mathbf{t}^{(\mathbf{i})})$ // $((n_{cm} + n_m) \times d)$
6: $\tilde{\mathbf{a}}^{(\mathbf{i})} \leftarrow \text{multihead\_attention}^{(i)}(\mathbf{h}^{(\mathbf{i})}, \mathbf{mem}_\mathbf{t}^{(\mathbf{i})})$ // MHA over both mem types $(n_s \times d)$
7: $\mathbf{a}^{(\mathbf{i})} \leftarrow \text{layer\_norm}(\tilde{\mathbf{a}}^{(\mathbf{i})} + \mathbf{h}^{(\mathbf{i})})$ // Regular skip + layernorm $(n_{cm} \times d)$
8: $\mathbf{old\_mem}^{(\mathbf{i})} \leftarrow \mathbf{m}_\mathbf{t}^{(\mathbf{i})}[: n_s]$ // Oldest memories to be forgotten $(n_s \times d)$
9: $\mathbf{new\_cm}^{(\mathbf{i})} \leftarrow f_c^{(i)}(\mathbf{old\_mem}^{(\mathbf{i})})$ // Compress oldest memories by factor $c$ $(\lfloor \frac{n_s}{c} \rfloor \times d)$
10: $\mathbf{m}_{\mathbf{t+1}}^{(\mathbf{i})} \leftarrow \text{concat}(\mathbf{m}_\mathbf{t}^{(\mathbf{i})}, \mathbf{h}^{(\mathbf{i})})[-n_m :]$ // Update memory $(n_m \times d)$
11: $\mathbf{cm}_\mathbf{t}^{(\mathbf{i})} \leftarrow \text{concat}(\mathbf{cm}_\mathbf{t}^{(\mathbf{i})}, \mathbf{new\_cm}^{(\mathbf{i})})[-n_{cm} :]$ // Update compressed memory $(n_{cm} \times d)$
12: $\mathbf{h}^{(\mathbf{i+1})} \leftarrow \text{layer\_norm}(\text{mlp}^{(i)}(\mathbf{a}^{(\mathbf{i})}) + \mathbf{a}^{(\mathbf{i})})$ // Mixing MLP $(n_s \times d)$

Compressive Transformers for Long-Range Sequence Modelling:
https://arxiv.org/abs/1911.05507

# Adaptive Memory Slot Allocation

Memory Demand:

- Extend the previous memory token for longer context
- Dynamically activate relevant memory slots
- Prevent memory interference via gated writing






**Researcher 1:** Binyan Sun

# Adaptive Memory Slot Allocation

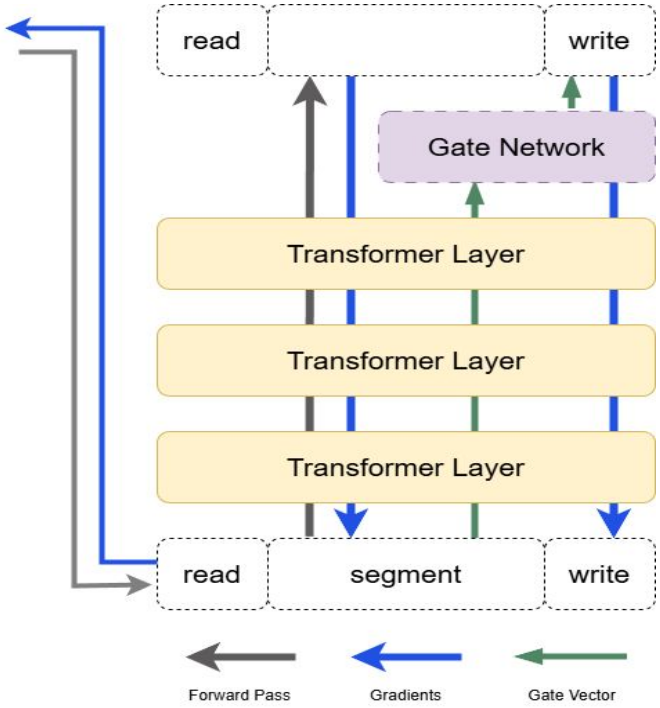
Gated Memory Tokens:

- Only the activated slots participate in effective writing and updating
- Gate vector computed from write state
- Selective memory slot activation
- Reduce cross-segment interference
- No modification to the Transformer architecture

$$g_\tau = \mathrm{softmax}\left(\mathrm{MLP}\left(H_\tau^{write}\right)\right)$$

$$\hat{H_\tau^{write}} = g_\tau \odot H_\tau^{write}$$

$$H_{\tau+1}^{\mathrm{mem}} = \mathrm{Update}\left(H_\tau^{\mathrm{mem}}, \hat{H}_\tau^{\mathrm{write}}\right)$$






**Researcher 1:** Binyan Sun

# Skip-Memory RMT (S-RMT)

We received the best perplexity with TR-XL + RMT

Here,

- RMT -> Long term memory
- TR-XL ->Short term memory

But, deeper BPTT unrolls causes instabilities and out-of-memory issues during RMT training for a larger memory sizes

Table 2: **Language modeling on WikiText-103.** Average perplexity for the best performed variations of RMT models reported (see full results in Appendix A.5). Underlined values show Tr-XL and RMT models with close results. RMT models with smaller memory sizes achieve similar scores to Tr-XL models with larger memory. Combination of cache with recurrent memory (Tr-XL + RMT) shows the best performance.

| MODEL | MEMORY | SEGMENT LEN | PPL$_{\pm STD}$ |
|---|---|---|---|
| TR-XL (PAPER) | 150 | 150 | 24.0 |
| BASELINE | 0 | 150 | 29.95 $\pm$ 0.15 |
| MEMTR | 10 | 150 | 29.63 $\pm$ 0.06 |
| TR-XL (OURS) | 150 | 150 | 24.12 $\pm$ 0.05 |
| TR-XL | 25 | 150 | 25.57 $\pm$ 0.02 |
| TR-XL | 75 | 150 | 24.68 $\pm$ 0.01 |
| RMT BPTT-3 | 10 | 150 | 25.04 $\pm$ 0.07 |
| RMT BPTT-2 | 25 | 150 | 24.85 $\pm$ 0.31 |
| TR-XL + RMT | 75+5 | 150 | 24.47 $\pm$ 0.05 |
| TR-XL + RMT | 150+10 | 150 | **23.99** $\pm$ 0.09 |
| BASELINE | 0 | 50 | 39.05 $\pm$ 0.01 |
| TR-XL | 100 | 50 | **25.66** $\pm$ 0.01 |
| TR-XL | 50 | 50 | 26.54 $\pm$ 0.01 |
| TR-XL | 25 | 50 | 27.57 $\pm$ 0.09 |
| TR-XL | 10 | 50 | 28.98 $\pm$ 0.11 |
| RMT BPTT-1 | 1 | 50 | 28.71 $\pm$ 0.03 |
| RMT BPTT-3 | 10 | 50 | 26.37 $\pm$ 0.01 |

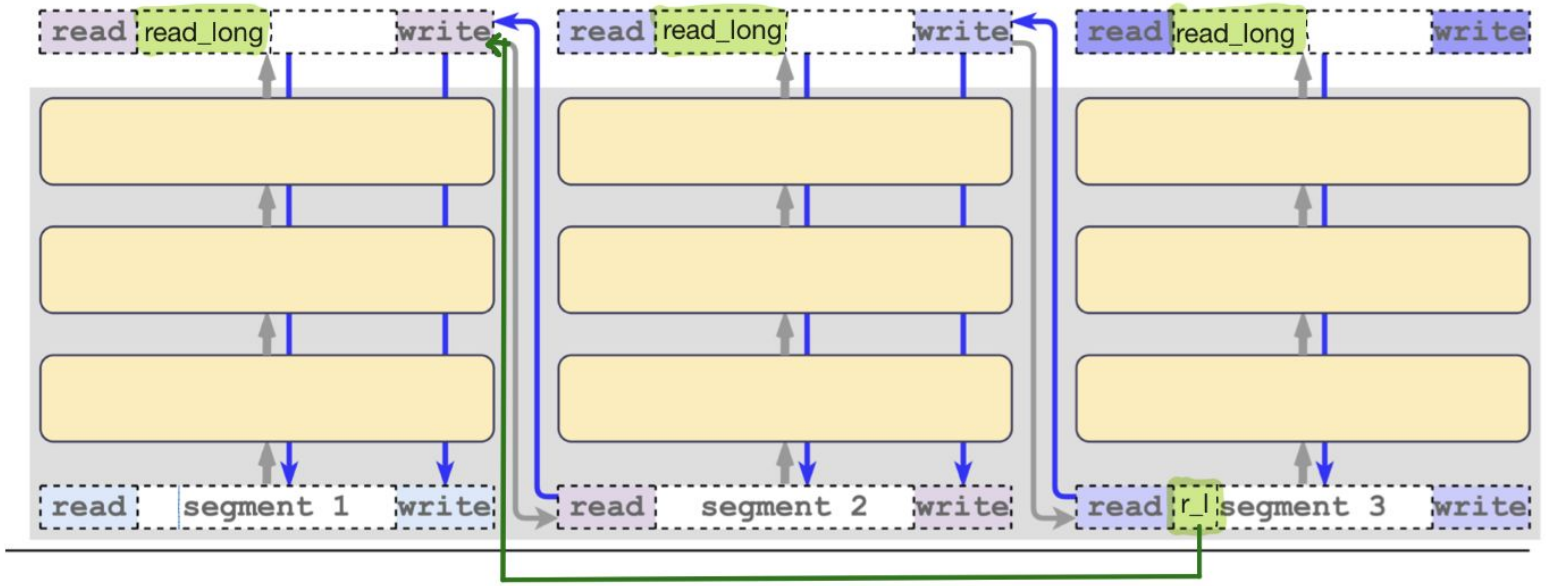


**Researcher 2:** Mihir Agarwal (ma4874)

# Skip-Memory RMT (S-RMT)

What if RMT handles both short term memory and long term memory.?

# Skip-Memory RMT (S-RMT)




Original - $$H_\tau = [H_{mem}; H_\tau^0; H_{mem}]$$

New - $$H_\tau = [H_{read\_recent}; H_{read\_long}; H_\tau^0; H_{write}]$$

# Skip-Memory RMT (S-RMT)

- $H_{read\_recent}$: Recurrent state from the immediate previous segment ($\tau - 1$).
- $H_{read\_long}$: Skip state from a distant previous segment ($\tau - k$), where $k$ is your dilation factor (e.g., $k = 50$).

## Update Rule

$$[H^N_{read\_recent}; H^N_{read\_long}; H^N_{\tau}; H^N_{write}] = \text{Transformer}(H_{\tau})$$

The update logic for the next segments follows:

1. **Working Memory Update**: $H^{\tau+1}_{read\_recent} = \text{Project}_{recent}(H^N_{write})$
2. **Long-Term Skip Update**: $H^{\tau+k}_{read\_long} = \text{Project}_{long}(H^N_{write})$


**Researcher 2:** Mihir Agarwal (ma4874)

# Skip-Memory RMT (S-RMT)

- $H_{read\_recent}$: Recurrent state from the immediate previous segment ($\tau - 1$).
- $H_{read\_long}$: Skip state from a distant previous segment ($\tau - k$), where $k$ is your dilation factor (e.g., $k = 50$).

## Update Rule

$$[H^N_{read\_recent}; H^N_{read\_long}; H^N_{\tau}; H^N_{write}] = \text{Transformer}(H_{\tau})$$

The update logic for the next segments follows:

1. **Working Memory Update**: $H^{\tau+1}_{read\_recent} = \text{Project}_{recent}(H^N_{write})$
2. **Long-Term Skip Update**: $H^{\tau+k}_{read\_long} = \text{Project}_{long}(H^N_{write})$

# Skip-Memory RMT (S-RMT)

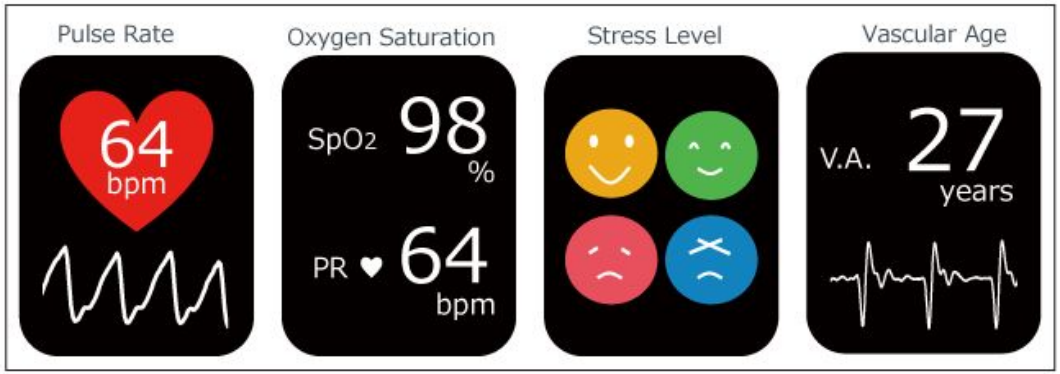
Benefits: -

1. Resolves Training Instability
2. Fully Differentiable Memory
3. Maintains Working memory and Global memory separate.
4. Reduced Recency Bias

# Edge AI for SmrtFone

- Goal: We want a low memory AI capable of running on device that can process data streams (heart rate, skin temp, sleep, etc.) from smart devices

- Issues with workarounds:
  - Chunking loses context
  - Offloading to cloud breaks privacy and adds extra latency



# Introducing ByteSized AI for SmrtFone

- ByteSized AI is developed using RMT
- RMT is a low memory augmentation to Transformer models
- We can use the data streams with RMT to help analyze and anticipate response based on User's health

# How to Deploy ByteSized AI

- RMT is trained off device to save computational complexity
- Sensor Data Rate: 10 Hz
- Input 5 second windows: ~50 tokens per input

# Things To Consider


- RMT is not optimal for general user use cases that doesn’t require serialized data
- Since it is mainly frozen on device, it needs a way to fine tune per user
- Due to limited memory, there is no mechanism to prevent forgetting of subtle or uncommon signals

The paper mentions using backpropagation through time to allow gradients to flow from the current segment through memory tokens to the previous segment. Figure 5 (in the paper) suggests that deeper BPTT improves perplexity but also introduces training instability. Given the known difficulty of training RNNs over long horizons, due to vanishing or exploding gradients, does the RMT attention mechanism mitigate signal degradation over many segments? Do memory tokens act as a memory highway similar to residual connections?

**Questions/Comments:** Ethan Simpson

The paper argues that deep BPTT is important because unrolling gradients into earlier segments improves scores (perplexity), but is that primarily because earlier segments learn to write better memory tokens? The improvements could also come from training with a longer recurrent rollout which increases the visible context. If the gains mainly come from training on longer rollouts rather than cross-segment gradient flow, is there a cheaper alternative to deep BPTT?

**Questions/Comments:** Layanne El Assaad

The paper stated that increasing memory has diminishing returns, where increasing memory size from 5 to 50 doesn’t add much. What determines the ‘optimal’ memory size in RMT?

**Questions/Comments:** Vici Milenia

I see the value in having a recurrent model embedding context into memory tokens, but I'm not convinced that each memory token helps maintain a sense of global context. I believe the memory tokens maintain a short-term context that represent something like a hidden state in an HMM, then the memory tokens can later be attended to by the transformer in a global manner, allowing for more efficient global search (yielding the long-term memory performance of RMT). I believe the way they describe the tokens as keeping long-term context may be somewhat misleading and better evaluation tasks could help to better shed light on how the method is actually working and what its limitations are.

**Questions/Comments:** Daniel Meyer

- Perplexity measures average prediction rather than specific recall, it does not show how well the past segments are memorized

- Increasing the memory size of Transformer-XL results in comparable performance (Table2), which means that brute-force caching is inefficient, and thus weaken the claim of superiority from the recurrent back-prop.